
Bayesian Linear Regression

ELG 5218 - Uncertainty Evaluation in Engineering Measurements and Machine Learning

Miodrag Bolic

Notation we will use throughout

- $\mathbf{x}_n \in \mathbb{R}^D$: the **input** (feature vector) of example n .
- $y_n \in \mathbb{R}$: its **target** (a real number).
- N : number of training examples; D : number of features.
- $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$: the training set.
- $\mathbf{w} \in \mathbb{R}^D$: the **weight vector** we want to learn.
- $\mathbf{X} = [\mathbf{x}_1 \cdots \mathbf{x}_N]^\top \in \mathbb{R}^{N \times D}$: the design matrix (one row per example).
- $\mathbf{y} = [y_1 \cdots y_N]^\top \in \mathbb{R}^N$: stacked targets.
- $\mathcal{N}(\cdot \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$: a Gaussian with mean $\boldsymbol{\mu}$, covariance $\boldsymbol{\Sigma}$.
- $\beta > 0$: **noise precision** (inverse variance of the observation noise).
- $\lambda > 0$: **regularization / prior precision** on the weights.
- $\mathbf{I}_D$: the $D \times D$ identity matrix.

A “precision” is just 1/variance. Large precision means small spread, i.e. high confidence.

What is regression?

Regression

A supervised-learning task in which we predict a **continuous** target $y \in \mathbb{R}$ from an input x , by learning a function f so that $y \approx f(x)$. (If y were a discrete label, the task would be *classification* instead.)

- We are given examples $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}$ and must predict y for *new* inputs.
- **Linear** regression restricts f to a linear function of the weights: $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} = \sum_{d=1}^D w_d x_d$.
- Concretely: predict a house price from its size and age; predict an exam score from hours studied. The weights say how strongly each feature moves the target.

Our whole question is: given the data, *which weights $\mathbf{w}$ should we believe in*, and how confident should we be?

The classical (non-Bayesian) answer: least squares

Choose the weights that make the total squared error on the training data as small as possible.

ordinary least squares (OLS)

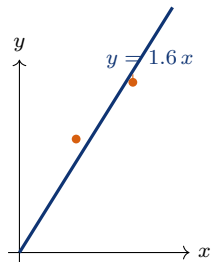
$$\mathbf{w}_{\text{LS}} = \arg \min_{\mathbf{w}} \sum_{n=1}^N (y_n - \mathbf{w}^\top \mathbf{x}_n)^2 = \arg \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2.$$

Setting the gradient to zero gives the closed form $\mathbf{w}_{\text{LS}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.

A small example we will reuse

One feature, model $y = wx$. Two points $(x_1, y_1) = (1, 2)$, $(x_2, y_2) = (2, 3)$. Then $\mathbf{X}^\top \mathbf{X} = 1^2 + 2^2 = 5$ and $\mathbf{X}^\top \mathbf{y} = 1 \cdot 2 + 2 \cdot 3 = 8$, so

$$w_{\text{LS}} = \frac{8}{5} = 1.60.$$



Where least squares gets into trouble

Least squares is fast and unbiased, but it can fail badly:

- **Overfitting.** With many features or few examples, it can fit the noise — driving some weights to large values and generalising poorly to new inputs.
- **Ill-conditioning.** If features are correlated (or $D > N$), $\mathbf{X}^\top \mathbf{X}$ is (near-)singular, so $(\mathbf{X}^\top \mathbf{X})^{-1}$ is unstable or does not exist.
- **No uncertainty.** It returns one line and nothing about how sure we are.

A standard cure is to *discourage large weights*. That idea is called regularization.

regularization (general)

Any technique that limits a model's **effective complexity** to reduce variance and improve generalization — trading a little training fit for out-of-sample performance. It need *not* be a term in the cost function.

Where it appears

- **Regression**: add $\lambda R(\mathbf{w})$ to the squared loss — ridge (ℓ_2), lasso (ℓ_1); convex, closed-form for ridge.
- **Deep networks**: some explicit (weight decay $= \ell_2$), but mostly *implicit / architectural* — dropout, early stopping, data augmentation, norm layers, weight sharing, SGD's own bias.

Not only a cost term (equivalent lenses)

- Penalty: $J(\mathbf{w}) = L(\mathbf{w}) + \lambda R(\mathbf{w})$
- Constraint: $\min L$ s.t. $\|\mathbf{w}\| \leq t$ (dual)
- Prior (MAP): Gaussian $\Leftrightarrow \ell_2$, Laplace $\Leftrightarrow \ell_1$
- Procedural: early stopping, dropout, augmentation

λ set by cross-validation / evidence — **not arbitrary**.

Types: **Shrinkage** (shrink weight magnitudes, keep all — ℓ_2 /ridge) · **Selection** (drive weights to 0, remove features — ℓ_1 /lasso, ARD) · **Structural / implicit** (architecture, dropout, early stopping).

Ridge regression

The most common regularizer penalizes the squared length of the weight vector.

ridge regression

$$\mathbf{w}_{\text{ridge}} = \arg \min_{\mathbf{w}} \underbrace{\|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2}_{\text{fit the data}} + \lambda \underbrace{\|\mathbf{w}\|^2}_{\text{keep } \mathbf{w} \text{ small}}, \quad \lambda \geq 0.$$

Closed form: $\mathbf{w}_{\text{ridge}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_D)^{-1} \mathbf{X}^\top \mathbf{y}.$

- The name: adding $\lambda \mathbf{I}_D$ places a “ridge” along the diagonal of $\mathbf{X}^\top \mathbf{X}$, which makes it **invertible and well-conditioned** even when OLS fails.
- $\lambda = 0$ recovers least squares; $\lambda \rightarrow \infty$ drives $\mathbf{w} \rightarrow \mathbf{0}$.

Same example, now with $\lambda = 1$

$$w_{\text{ridge}} = \frac{\mathbf{X}^\top \mathbf{y}}{\mathbf{X}^\top \mathbf{X} + \lambda} = \frac{8}{5 + 1} = \frac{4}{3} \approx \mathbf{1.33} \text{ — shrunk from 1.60 toward 0.}$$

Why would we use λ ?

The single number λ controls how much we trust the data versus our preference for small weights.

- **Bias–variance trade-off.** A larger λ adds a little bias but reduces variance; for a suitable λ the predictions generalise better than OLS.
- **Stability.** It guarantees $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ is invertible, curing the ill-conditioning from the previous slide.
- **Small-data / high-dimensional regimes.** When N is small or D is large, shrinkage is what keeps the fit sensible.

How do we pick λ ? Classically, by **cross-validation**. Later in this lecture we will see that the Bayesian view chooses it from the data automatically (via the evidence). The rest of the lecture shows that ridge regression is not an ad-hoc trick: it is exactly what a Gaussian prior on $\mathbf{w}$ gives you.

Step 1: a probabilistic model for the data (the likelihood)

Assume each target is the linear prediction plus independent Gaussian noise:

$$y_n = \mathbf{w}^\top \mathbf{x}_n + \epsilon_n, \quad \epsilon_n \sim \mathcal{N}(0, \beta^{-1}) \implies p(y_n \mid \mathbf{x}_n, \mathbf{w}, \beta) = \mathcal{N}(y_n \mid \mathbf{w}^\top \mathbf{x}_n, \beta^{-1}).$$

Collecting all N i.i.d. examples gives the **likelihood**

$$p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}, \beta) = \mathcal{N}(\mathbf{y} \mid \mathbf{X}\mathbf{w}, \beta^{-1}\mathbf{I}_N) \propto \exp\left[-\frac{\beta}{2}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2\right].$$

- Its negative log is $\frac{\beta}{2}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \text{const}$ — the **sum of squared errors**. So maximising the likelihood *is* least squares: $\mathbf{w}_{\text{MLE}} = \mathbf{w}_{\text{LS}}$.
- We have turned “minimise squared error” into “assume Gaussian noise.”

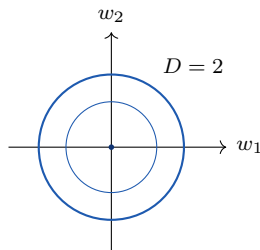
Step 2: a prior over the weights

Before seeing data we express a mild preference for small weights as a Gaussian prior.

the (isotropic) weight prior

$$p(\mathbf{w}) = \prod_{d=1}^D \mathcal{N}(w_d \mid 0, \lambda^{-1}) = \mathcal{N}(\mathbf{w} \mid \mathbf{0}, \lambda^{-1} \mathbf{I}_D).$$

- **Dimension.** $p(\mathbf{w})$ is a D -dimensional Gaussian — one axis per weight. This is a distribution over the whole vector $\mathbf{w}$, not over a single number.
- **“Common” / isotropic prior.** A single shared precision λ , zero mean, diagonal covariance: all D axes have equal width and the contours are spheres. Each weight is a priori independent with the same spread $\lambda^{-1/2}$.
- Large λ = strong belief the weights are near 0.



Spherical contours: the isotropic prior.

Ridge regression is MAP estimation

Combine the two pieces with Bayes' rule; the most probable weights (the MAP estimate) maximise

$$\log p(\mathbf{w} \mid \mathcal{D}) = \underbrace{-\frac{\beta}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2}_{\text{from the likelihood}} \underbrace{-\frac{\lambda}{2} \|\mathbf{w}\|^2}_{\text{from the prior}} + \text{const.}$$

- Maximising this is *exactly* the ridge objective (with regularization strength λ/β). So:

$$\text{ridge regression} = \text{MAP estimate under a Gaussian prior}$$

- The penalty $-\log p(\mathbf{w}) \propto \|\mathbf{w}\|^2$ is where the “ ℓ_2 ” term comes from.
- But a point estimate still throws away the *shape* of the belief. Keeping the whole posterior (not just its peak) is what makes the method **Bayesian** — and gives us uncertainty.

Priors with structure: ARD and sparsity

We can give *each* weight its own precision instead of one shared λ :

$$p(\mathbf{w} \mid \{\lambda_d\}) = \prod_{d=1}^D \mathcal{N}(w_d \mid 0, \lambda_d^{-1}).$$

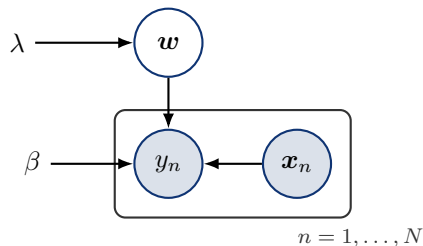
Automatic Relevance Determination (ARD)

Learn each λ_d from the data. If feature d is irrelevant, its precision grows ($\lambda_d \rightarrow \infty$), forcing $w_d \rightarrow 0$ and effectively **switching the feature off**. The model thus “determines the relevance” of each feature by itself.

sparsity-promoting prior

A prior under which most weights are pushed to (near) zero, so the fitted model uses only a **few** features. “Sparse” means few non-zero entries. ARD is one route; heavy-tailed priors are another.

The full model, in plate notation



- Shaded nodes are *observed* (y_n, x_n); the open node w is the *unknown* we infer.
- The plate means “repeat for every example n ”: all N targets share the same weights w .
- λ, β are fixed hyperparameters (for now).
- Reading the graph:
 $p(w \mid \lambda) \prod_n p(y_n \mid x_n, w, \beta)$.

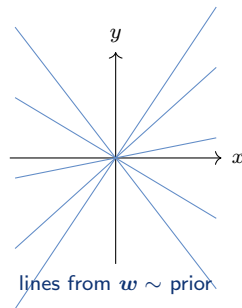
This picture is the model. Inference means turning the arrows around: given the shaded nodes, what is w ?

First, the model as a generative story

Before we touch data, our model says how the world produces (x, y) pairs:

- 1 Draw one weight vector from the prior: $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \lambda^{-1} \mathbf{I}_D)$.
- 2 For each input $\mathbf{x}_n$, draw a target with Gaussian noise: $y_n \sim \mathcal{N}(\mathbf{w}^\top \mathbf{x}_n, \beta^{-1})$.

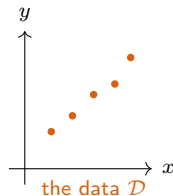
- With *no* data, our belief about $\mathbf{w}$ is the prior — a broad Gaussian centred at $\mathbf{0}$.
- Each candidate $\mathbf{w}$ corresponds to one line. Sampling $\mathbf{w}$ from the prior gives a spread of plausible lines (right).



Now bring in the data

We observe the training set $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$.

- Each observed point rules out lines that miss it, and rewards lines that pass near it.
- In the likelihood, this is the factor $p(y_n | \mathbf{x}_n, \mathbf{w}) = \mathcal{N}(y_n | \mathbf{w}^\top \mathbf{x}_n, \beta^{-1})$: a soft constraint on $\mathbf{w}$.
- Combining the prior with all these constraints (Bayes' rule) gives the **posterior** over $\mathbf{w}$ — the subject of the next slide.



The plan: prior (broad belief) + data (constraints) $\Rightarrow$ posterior (updated belief). We compute it in closed form because everything is Gaussian.

The posterior over the weights

Because a Gaussian prior meets a Gaussian likelihood, the posterior is again Gaussian. This is precisely the “posterior of $\mathbf{x}$ given $\mathbf{y}$ ” result from our Gaussian handout, with the weights $\mathbf{w}$ playing the role of the hidden variable.

Corollary 1 (Gaussian posterior), applied to $\mathbf{w}$

$$p(\mathbf{w} \mid \mathcal{D}) = \mathcal{N}(\mathbf{w} \mid \boldsymbol{\mu}_N, \boldsymbol{\Sigma}_N), \quad \boxed{\boldsymbol{\Sigma}_N = (\lambda \mathbf{I}_D + \beta \mathbf{X}^\top \mathbf{X})^{-1}, \quad \boldsymbol{\mu}_N = \boldsymbol{\Sigma}_N \beta \mathbf{X}^\top \mathbf{y}}.$$

- The prior precision $\lambda \mathbf{I}_D$ and the data precision $\beta \mathbf{X}^\top \mathbf{X}$ simply **add** inside $\boldsymbol{\Sigma}_N^{-1}$ — more data means a tighter posterior.
- The posterior *mean* equals the ridge solution with λ/β : the MAP point sits at $\boldsymbol{\mu}_N$. The Bayesian answer keeps the whole $\mathcal{N}(\boldsymbol{\mu}_N, \boldsymbol{\Sigma}_N)$, not just its peak.

Where the weight-posterior formula comes from

Write the regression model as a linear–Gaussian instance of the handout:

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{w} \mid \mathbf{0}, \lambda^{-1} \mathbf{I}_D), \quad p(\mathbf{y} \mid \mathbf{w}) = \mathcal{N}(\mathbf{y} \mid \mathbf{X}\mathbf{w}, \beta^{-1} \mathbf{I}_N).$$

Match to the affine model $p(\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_x, \boldsymbol{\Sigma}_x)$, $p(\mathbf{y} \mid \mathbf{x}) = \mathcal{N}(A\mathbf{x} + b, \boldsymbol{\Sigma}_{y|x})$:

$$\mathbf{x} \rightarrow \mathbf{w}, \quad A = \mathbf{X}, \quad b = \mathbf{0}, \quad \boldsymbol{\Sigma}_x = \lambda^{-1} \mathbf{I}_D, \quad \boldsymbol{\Sigma}_{y|x} = \beta^{-1} \mathbf{I}_N.$$

Step 1 (Theorem 3). A Gaussian prior through an affine–Gaussian likelihood makes $(\mathbf{w}, \mathbf{y})$ **jointly Gaussian**.

Step 2 (condition on $\mathbf{y}$: Theorem 2 = Corollary 1). Substitute the matches above:

$$\boldsymbol{\Sigma}_N = (\boldsymbol{\Sigma}_x^{-1} + A^\top \boldsymbol{\Sigma}_{y|x}^{-1} A)^{-1} = (\lambda \mathbf{I}_D + \mathbf{X}^\top (\beta \mathbf{I}_N) \mathbf{X})^{-1} = (\lambda \mathbf{I}_D + \beta \mathbf{X}^\top \mathbf{X})^{-1},$$

$$\boldsymbol{\mu}_N = \boldsymbol{\Sigma}_N (\boldsymbol{\Sigma}_x^{-1} \boldsymbol{\mu}_x + A^\top \boldsymbol{\Sigma}_{y|x}^{-1} (\mathbf{y} - b)) = \boldsymbol{\Sigma}_N (\lambda \mathbf{I}_D \mathbf{0} + \mathbf{X}^\top \beta \mathbf{I}_N \mathbf{y}) = \boldsymbol{\Sigma}_N \beta \mathbf{X}^\top \mathbf{y}.$$

Result: Corollary 1 applied to $\mathbf{w}$

$$p(\mathbf{w} \mid \mathcal{D}) = \mathcal{N}(\mathbf{w} \mid \boldsymbol{\mu}_N, \boldsymbol{\Sigma}_N), \quad \boldsymbol{\Sigma}_N = (\lambda \mathbf{I}_D + \beta \mathbf{X}^\top \mathbf{X})^{-1}, \quad \boldsymbol{\mu}_N = \boldsymbol{\Sigma}_N \beta \mathbf{X}^\top \mathbf{y}.$$

Precision adds: $\boldsymbol{\Sigma}_N^{-1} = \underbrace{\lambda \mathbf{I}_D}_{\text{prior}} + \underbrace{\beta \mathbf{X}^\top \mathbf{X}}_{\text{data}}$ (always invertible, so the posterior is proper even when $\mathbf{X}^\top \mathbf{X}$ is

singular). **MAP = ridge:** $\boldsymbol{\mu}_N = (\mathbf{X}^\top \mathbf{X} + \frac{\lambda}{\beta} \mathbf{I}_D)^{-1} \mathbf{X}^\top \mathbf{y}.$

Our running example, with $\lambda = 1$, $\beta = 1$

Recall $\mathbf{X}^\top \mathbf{X} = 5$ and $\mathbf{X}^\top \mathbf{y} = 8$. Then

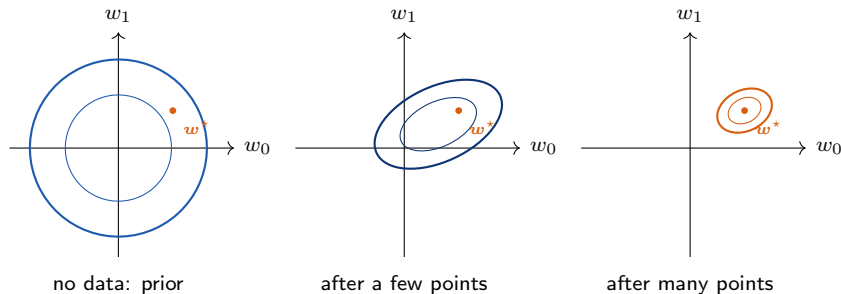
$$\Sigma_N = (\lambda + \beta \mathbf{X}^\top \mathbf{X})^{-1} = (1 + 5)^{-1} = \frac{1}{6}, \quad \mu_N = \Sigma_N \beta \mathbf{X}^\top \mathbf{y} = \frac{1}{6} \cdot 8 = \frac{4}{3}.$$

$$\boxed{w \mid \mathcal{D} \sim \mathcal{N}\left(\frac{4}{3}, \frac{1}{6}\right)} \quad (\text{mean} \approx 1.33, \text{ sd} \approx 0.41).$$

- The mean 1.33 is the ridge estimate we computed earlier — consistent, as expected.
- We now also have a spread ($\text{sd} \approx 0.41$): a statement of *how sure* we are about the slope. Least squares never gave us this.

Updating our belief: the posterior in weight space

With the model and data in hand, watch the belief about w tighten as examples arrive.



- We start at the prior (broad, centred at $\mathbf{0}$). Each observation multiplies in a likelihood factor, so the posterior **concentrates** around the true weights w^* .
- The ellipse tilts because correlated features give correlated weight estimates — this correlation is exactly the off-diagonal of Σ_N .

Predicting at a new input

For a test input $\mathbf{x}_*$ we do not commit to one $\mathbf{w}$; we average the predictions of *all* weights, weighted by the posterior. This is again a Gaussian identity (marginal of y).

Corollary 2 (predictive distribution), applied to w

$$p(y_* | \mathbf{x}_*, \mathcal{D}) = \mathcal{N}(\mathbf{x}_*^\top \boldsymbol{\mu}_N, \beta^{-1} + \mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_*)$$

- Predictive **mean** $\mathbf{x}_*^\top \boldsymbol{\mu}_N$: the prediction of the posterior-mean weights.
- Predictive **variance** has two parts: the irreducible noise β^{-1} , plus $\mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_*$, our uncertainty about $\mathbf{w}$ carried through to $\mathbf{x}_*$.
- A plug-in estimate ($\mathcal{N}(\hat{\mathbf{w}}^\top \mathbf{x}_*, \beta^{-1})$) keeps only the first part and is therefore over-confident.

Where the predictive formula comes from

For a new input the **posterior now plays the role of the prior** over $\mathbf{w}$, and the observation is again affine-Gaussian:

$$p(\mathbf{w} \mid \mathcal{D}) = \mathcal{N}(\mathbf{w} \mid \boldsymbol{\mu}_N, \boldsymbol{\Sigma}_N), \quad y_* = \mathbf{x}_*^\top \mathbf{w} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \beta^{-1}) \Rightarrow p(y_* \mid \mathbf{w}) = \mathcal{N}(\mathbf{x}_*^\top \mathbf{w}, \beta^{-1}).$$

Match to the affine model $p(\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_x, \boldsymbol{\Sigma}_x)$, $p(y \mid \mathbf{x}) = \mathcal{N}(A\mathbf{x} + b, \boldsymbol{\Sigma}_{y|x})$:

$$\mathbf{x} \rightarrow \mathbf{w}, \quad \boldsymbol{\mu}_x = \boldsymbol{\mu}_N, \quad \boldsymbol{\Sigma}_x = \boldsymbol{\Sigma}_N, \quad A = \mathbf{x}_*^\top, \quad b = 0, \quad \boldsymbol{\Sigma}_{y|x} = \beta^{-1}.$$

Step 1 (Theorem 3). $(\mathbf{w}, y_*)$ are **jointly Gaussian**.

Step 2 (marginalize out $\mathbf{w}$: Theorem 1 = Corollary 2). Substitute the matches above:

$$\text{mean} = A\boldsymbol{\mu}_x + b = \mathbf{x}_*^\top \boldsymbol{\mu}_N,$$

$$\text{var} = \boldsymbol{\Sigma}_{y|x} + A\boldsymbol{\Sigma}_x A^\top = \beta^{-1} + \mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_*.$$

Result: Corollary 2 applied to $\mathbf{w}$

$$p(y_* \mid \mathbf{x}_*, \mathcal{D}) = \mathcal{N}(\mathbf{x}_*^\top \boldsymbol{\mu}_N, \beta^{-1} + \mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_*).$$

The two variance parts are exactly Corollary 2's $\boldsymbol{\Sigma}_{y|x} + A\boldsymbol{\Sigma}_x A^\top$: **irreducible noise** β^{-1} plus **weight uncertainty** $\mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_*$ carried through $\mathbf{x}_*$. A plug-in $\mathcal{N}(\hat{\mathbf{w}}^\top \mathbf{x}_*, \beta^{-1})$ drops the second term ($\boldsymbol{\Sigma}_x \rightarrow 0$) and is over-confident.

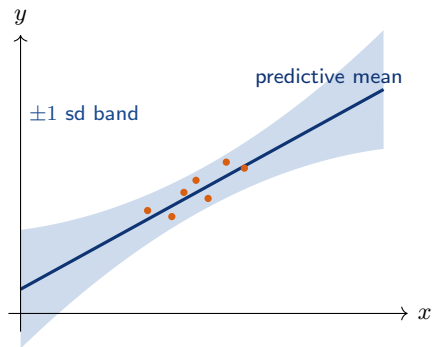
Predict at $x_* = 3$ (same $\mu_N = \frac{4}{3}$, $\Sigma_N = \frac{1}{6}$, $\beta = 1$)

$$\text{mean} = x_* \mu_N = 3 \cdot \frac{4}{3} = 4, \quad \text{variance} = \beta^{-1} + x_*^2 \Sigma_N = 1 + 9 \cdot \frac{1}{6} = \frac{5}{2} = 2.5.$$

$$y_* \mid x_* = 3 \sim \mathcal{N}(4, 2.5) \quad (\text{sd} \approx 1.58).$$

- The variance grows with x_*^2 : we are **least certain far from the training data**. Near the data (e.g. $x_* = 1$) it is only $1 + \frac{1}{6} \approx 1.17$.
- That growing error bar is the practical payoff of being Bayesian.

The predictive error bar in a picture



- The band width at x is $\sqrt{\beta^{-1} + \mathbf{x}^\top \Sigma_N \mathbf{x}}$: narrow where data is dense, wide where we must extrapolate.
- A least-squares fit would draw a band of *constant* width — over-confident precisely where it should hesitate.

Two kinds of uncertainty, in one formula

The predictive variance we just derived splits cleanly into two pieces:

$$\text{Var}[y_* \mid \mathbf{x}_*, \mathcal{D}] = \underbrace{\beta^{-1}}_{\text{aleatoric}} + \underbrace{\mathbf{x}_*^\top \Sigma_N \mathbf{x}_*}_{\text{epistemic}}.$$

aleatoric (aleatory) uncertainty

Irreducible randomness in the data itself — here the observation noise β^{-1} . Collecting more of the *same* data will not shrink it; it is a property of the world / measurement, not of our ignorance.

epistemic uncertainty

Uncertainty about the *parameters*: the posterior spread Σ_N . It is reducible — $\Sigma_N = (\lambda \mathbf{I} + \beta \mathbf{X}^\top \mathbf{X})^{-1}$ shrinks roughly like $1/N$, and $\mathbf{x}_*^\top \Sigma_N \mathbf{x}_*$ grows where data are sparse.

A third kind — **model / structural** uncertainty (is the linear-Gaussian form even right?) — is *not* captured. BLR is confident *within* its assumptions; closing that gap is what the diagnostics later in the lecture are for.

Reducing each kind of uncertainty

The two respond to different levers, so it pays to know which one you are looking at.

Epistemic — reducible with data

- Collect *more* data.
- Collect *informative* data: active learning / optimal design query inputs where $x_*^\top \Sigma_N x_*$ is largest.
- Spread out and decorrelate the inputs — better conditioning of $X^\top X$ tightens Σ_N .
- Use an informative prior when data are scarce (at the cost of some bias).
- Use fewer / sparser features (lower variance).

Aleatoric — not fixed by more data

- Improve the *measurement*: better instruments, cleaner labels.
- Add covariates that explain what was previously “noise.”
- If the noise is input-dependent, *represent* it with a heteroscedastic model instead of a single β .

Setting β, λ well (via the evidence) is what makes the reported uncertainties *calibrated*, rather than merely large or small.

Recall we deferred the question of how to set λ . The Bayesian answer uses the **marginal likelihood** (the “evidence”), which is once more a Gaussian:

$$p(\mathbf{y} \mid \mathbf{X}, \lambda, \beta) = \int p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}, \beta) p(\mathbf{w} \mid \lambda) d\mathbf{w} = \mathcal{N}(\mathbf{y} \mid \mathbf{0}, \beta^{-1} \mathbf{I}_N + \lambda^{-1} \mathbf{X} \mathbf{X}^\top).$$

- Maximising this over (λ, β) selects the regularization strength *without* a validation set — the data tell us how much to shrink.
- If we instead give every weight its own λ_d and maximise the evidence over all of them, irrelevant features are driven off automatically. That procedure is exactly the ARD we defined earlier.

When is Bayesian linear regression the right tool?

The model is trustworthy to the extent its assumptions hold:

- 1 **Linear in the parameters**: $\mathbb{E}[y | \mathbf{x}] = \mathbf{w}^\top \phi(\mathbf{x})$ (the features may be nonlinear in $\mathbf{x}$).
- 2 **Additive Gaussian noise**, $\epsilon \sim \mathcal{N}(0, \beta^{-1})$ — i.e. a Gaussian likelihood.
- 3 **Homoscedasticity**: constant noise variance across inputs (unless we extend the model).
- 4 **Independent errors** given $\mathbf{w}$ (no autocorrelation).
- 5 **Correct specification**: mean-zero errors, no omitted variable correlated with $\mathbf{x}$, inputs measured with negligible error.
- 6 A **sensible prior** on $\mathbf{w}$ (Gaussian for the closed form), with β, λ known or given hyperpriors.

When these hold, the closed-form posterior and predictive are exact and the credible intervals mean what they say.

When to be cautious — or use something else

- **Genuine nonlinearity** basis functions can't capture → Gaussian processes, splines, neural nets.
- **Heavy tails / outliers** → the Gaussian likelihood is non-robust; use Student- t , Huber, or quantile regression.
- **Heteroscedastic noise** → a single β makes intervals too wide in some regions and too narrow in others; model input-dependent noise.
- **Bounded / positive / count / categorical targets** → use the matching GLM or a censored / truncated model.
- **Correlated errors** (time series, spatial) → independence is violated and you will *understate* uncertainty; add temporal / spatial structure.
- **Noisy inputs** (errors-in-variables) → coefficients are biased; use total-least-squares.
- **Far extrapolation, or causal claims from confounded data** → structural error and confounding are not represented.

The one caution to remember

BLR quantifies uncertainty *inside* its assumed model. Under misspecification it will hand you tight, confident, *wrong* intervals — which is why we run the checks that follow.

Look at both the parameter posterior and the predictive distribution.

On the weights

- Report μ_N with per-coefficient credible intervals from $\sqrt{\text{diag}(\Sigma_N)}$.
- An interval that excludes 0 $\Rightarrow$ that coefficient is credibly non-zero.
- Off-diagonals of Σ_N : strong posterior correlations mean features trade off and are not individually identifiable.

On the predictions

- **Posterior predictive checks**: simulate $\tilde{\mathbf{y}} \sim p(\tilde{\mathbf{y}} \mid \mathcal{D})$ and compare its summaries to the real data.
- **Calibration**: do your 90% intervals cover $\approx 90\%$ of held-out points?
- **Prior sensitivity**: vary λ ; if conclusions swing, you are prior-dominated.
- **Model comparison**: evidence / Bayes factors, LOO-CV, WAIC — not in-sample fit.

Score the point prediction *and* the whole distribution — and lean on the latter, since a point-only metric throws away the reason we went Bayesian.

Point accuracy

(on the mean $\mathbf{x}_*^\top \boldsymbol{\mu}_N$)

- RMSE, MAE
- R^2 / adjusted R^2

Probabilistic quality (the important ones)

- **NLPD** (neg. log predictive density) — primary; punishes wrong means *and* mis-sized variances.
- **CRPS** — a proper score over the whole predictive CDF.
- **Coverage / PICP** with mean interval width — sharp *subject to* being calibrated.
- **PIT histograms** / reliability diagrams.

Rule of thumb

A model can win on RMSE and still be badly calibrated; proper scoring rules (NLPD, CRPS) catch that, RMSE alone does not. For model selection use evidence / PSIS-LOO / WAIC.

Reading the residuals

Residual $r_n = y_n - \hat{y}_n$ with $\hat{y}_n = \boldsymbol{\mu}_N^\top \mathbf{x}_n$. Prefer the **standardized** residual

$$\tilde{r}_n = \frac{y_n - \boldsymbol{\mu}_N^\top \mathbf{x}_n}{\sqrt{\beta^{-1} + \mathbf{x}_n^\top \boldsymbol{\Sigma}_N \mathbf{x}_n}},$$

which folds in the predictive uncertainty. Each plot tests one assumption:

- **Residuals vs. fitted**: want a flat band; curvature $\rightarrow$ nonlinearity, funnel $\rightarrow$ heteroscedasticity.
- **Residuals vs. each predictor**: shows *which* feature needs a nonlinear term.
- **Q-Q vs. Normal**: checks the Gaussian-noise assumption; heavy tails / skew show here.
- **Scale–location**: a cleaner view of non-constant variance.
- **Residuals vs. time / order** (ACF): detects autocorrelation.
- **Leverage & Cook's distance**: flags influential points.

The Bayesian calibration check

If the noise model is right, $\tilde{r}_n \sim \mathcal{N}(0, 1)$ — in particular their *variance should be* ≈ 1 . Variance > 1 : β too large (noise underestimated); < 1 : intervals too wide. General version: each point's PIT should be uniform (ideally using LOO residuals).

What to do about what you find

Residual analysis is not a report card — it is the input to the next model, and it maps back to the two uncertainties:

- **Curvature / pattern** → add features, interactions, or basis functions; if that is not enough, move to a GP or nonlinear model. (*fixing structural error*)
- **Heteroscedasticity** → model input-dependent noise, transform y (log / Box–Cox), or weight the fit. (*getting the aleatoric term right*)
- **Heavy tails / outliers** → switch to a Student- t or Huber likelihood; check influence *before* deleting anything.
- **Autocorrelation** → add temporal / spatial structure (AR terms, a GP kernel over time).
- **Standardized-residual variance $\neq 1$** → re-estimate β (evidence maximization); this recalibrates every predictive interval.
- **Residuals tracking an omitted variable** → you have likely found a missing confounder; add it.

Then refit and recheck. The loop *fit* → *check* → *revise* is the real practice of regression, Bayesian or not.

Replace the raw input by a fixed set of **basis functions** $\phi(\mathbf{x}) = [\phi_1(\mathbf{x}), \dots, \phi_M(\mathbf{x})]^\top$:

$$y = \mathbf{w}^\top \phi(\mathbf{x}) + \epsilon, \quad \phi(x) = [1, x, x^2, \dots]^\top \text{ (for example).}$$

- The model is still *linear in the weights* $\mathbf{w}$, so every formula we derived holds verbatim — just swap $\mathbf{X}$ for the matrix Φ with rows $\phi(\mathbf{x}_n)^\top$.
- We can fit curves (polynomials, splines, Gaussian bumps) while keeping a closed-form Gaussian posterior and predictive.
- Question this raises: what if we want *very many* basis functions — even infinitely many? That leads to Gaussian processes.

The wider family: change one ingredient

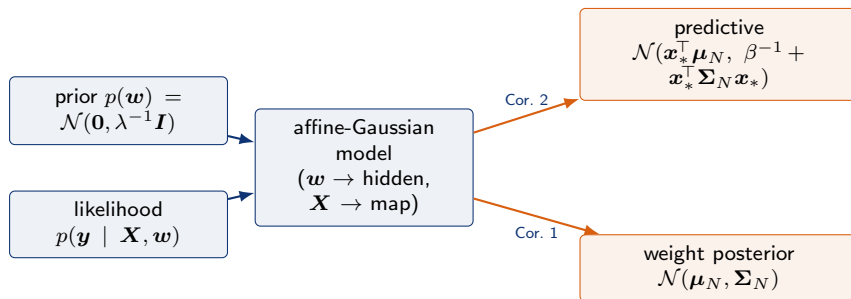
Almost every named regression method is this same model with *one* axis changed.

Change...	...and you get	Relation to linear regression
Regularizer / prior	ridge (L2 / Gaussian), Lasso (L1 / Laplace, sparse), elastic net	same model, different prior — all are MAP estimates
Functional form	polynomials & basis functions; kernel ridge, SVR, Gaussian processes; neural nets, trees, forests, boosting	basis: identical math; kernels: implicit features; nets: nonlinear in w
Noise / loss	GLMs (logistic, Poisson, Gamma) via a link; quantile; robust (Huber, Student- t)	linear regression = Gaussian response, identity link
Structure	hierarchical / mixed-effects; PCR, PLS	shared or reduced weights
Estimate	point (MLE / MAP) vs. full posterior	Bayesian keeps the whole $\mathcal{N}(\mu_N, \Sigma_N)$

Linear regression is the hub: pick an axis, change it, and you land on a named method. Several “changes” compose — e.g. a Bayesian, sparse, kernelized model.

The big picture: one set of Gaussian rules

Everything today came from two operations on Gaussians, applied with $\mathbf{w}$ as the hidden variable and the design matrix $\mathbf{X}$ as the linear map.



The prior and likelihood assemble into one affine-Gaussian model; conditioning gives the weight posterior, and marginalising gives the predictive. The same two rules will return for Gaussian processes and for the Kalman filter.

- **Regression** predicts a continuous target; **least squares** is the classical fit, but it can overfit, be unstable, and reports no uncertainty.
- **Ridge regression** adds an ℓ_2 penalty controlled by λ ; it is exactly **MAP estimation** under a Gaussian prior on the weights.
- The full Bayesian treatment keeps the whole **posterior** $\mathcal{N}(\boldsymbol{\mu}_N, \boldsymbol{\Sigma}_N)$ and the **predictive** $\mathcal{N}(\mathbf{x}_*^\top \boldsymbol{\mu}_N, \beta^{-1} + \mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_*)$ — giving an input-dependent error bar.
- Structured priors (**ARD**, sparsity) select features; the **evidence** chooses λ, β from the data.
- The predictive variance separates **aleatoric** noise β^{-1} from **epistemic** spread $\mathbf{x}_*^\top \boldsymbol{\Sigma}_N \mathbf{x}_*$; only the latter shrinks with data.
- BLR is exact *under its assumptions* — check them with residuals and calibration, and score with proper rules (NLPD, CRPS), not RMSE alone.
- Change one ingredient (prior, features, noise model) and you reach the rest of the regression family; the infinite-basis limit is a **Gaussian process**.

- C. Bishop, *Pattern Recognition and Machine Learning* (2006), Ch. 3 — linear regression, ridge, and the Bayesian treatment.
- K. Murphy, *Probabilistic Machine Learning: An Introduction* (2022) — linear models and MAP/ridge.
- D. MacKay, *Information Theory, Inference, and Learning Algorithms* (2003) — the evidence framework for choosing hyperparameters.
- C. Rasmussen & C. Williams, *Gaussian Processes for Machine Learning* (2006) — the function-space view we previewed at the end.

Companion handout: *Gaussian Identities with Examples* (the marginal, conditional, affine, and posterior/predictive results used throughout).